In Figure 2 (a), we compare the MLP-Mixer with $S = C = 64, 32$ and the SW-MLP. The sparsity of the SW-MLP is taken as 2^{-n} where n ranges from 0 to 10. We set the patch size as 4. Each network is trained on CIFAR10 with a batch size of 128, for 600 epochs, a learning rate of 0.01, using auto-augmentation, AdamW optimizer, momentum set to 0.9, and cosine annealing. We utilize three different random seeds for each training.

Figure 2(b) shows the CKA (for a specific seed) of the MLP-Mixer with $S = C = 64$ and the MLP with a sparsity of 1/64, based on the results from Figure 2(a). However, the features targeted for CKA are taken from the layer just before the skip-connection in each block, and they are displayed on the axis in the order of proximity to the input, labeled as 1 through 6. Figure 2(c) similarly compares the dense MLP (i.e. sparsity$= 1$).

Figure 2(d) shows the test error of the shallow MLP with monarch matrix weight and Kronecker weight matrix. The training uses MNIST, with optimizer adamw with 200 epochs, and the cosine annealing of leaning rate with an initial value of 0.01. Table S.1 summarizes the difference of the shallow models between other treated models.

C.2. Figure 4

In Figure 4 (left), we compare the MLP-Mixer and the MLP under the same training settings as in Figure 2(a). Now, here $\Omega = 2^{19}$ and $\gamma = 2$, with C values being 4,8,12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 64. The S values are determined corresponding to C while keeping Ω fixed, resulting in values: 360, 252, 203, 173, 152, 136, 124, 113, 104, 96, 89, 83, 64. We utilized four different random seeds for each training.

In Figure 4 (right), we used the same values on Ω, γ and C, S as the Figure 4 (left). We run 10 trials with different random seeds to plot the singular values of the sparse weight matrix.

C.3. Table 1

CIFAR10, CIFAR100 We used single Tesla V100 GPU to compute the runtime. The training is by AdamW with the mini-batch size 128. We used a 32-bit float. The runtime is averaged over 600 epochs. The averaged runtime is on four different random seeds.

For our experiments, we utilized Tesla V100 GPUs, accumulating approximately 300 GPU hours. The networks were trained on datasets, either CIFAR-10 or CIFAR-100. We set $L = 2$ and $\gamma = 4$ for the Mixer-SS/8 and Mixer-SS-W. For the Mixer-SS/8 configuration, the parameters were set as $p = 8$, $S = 16$, and $C = 986$. In our wider setting (Mixer-SS-W), the parameters were defined as $p = 4$, $S = 64$, and $C = 487$. The training was conducted over 4000 epochs with the cosine-annealing. We employed the AdamW optimizer and incorporated auto-augmentation techniques. The chosen mini-batch size was 1024, and experiments were run with three distinct random seeds. To optimize results, we conducted a hyper-parameter search for the best initial learning rate from the set $\{0.04, 0.05, 0.06\}$. The rate that provided the highest accuracy, when averaged over the random seeds, was subsequently chosen for experiments. Specifically, the learning rate 0.06 was selected for Mixer-SS/8 on CIFAR-100, while 0.04 was the preferred rate for all other scenarios.

ImageNet-1k For the Mixer-B-W, we set $L = 8, p = 14, C = 588, S = 256$, and $\gamma = 4.57$. The other settings are the same as the other experiments on ImgeNet-1k (Appendix C.5).

C.4. Figure 5

We utilized Tesla V100 GPUs and approximately 400 GPU hours for this experiment. We trained three types of MLPs; S-Mixer, RP S-Mixer, and SW-MLP architectures. All MLPs incorporated a per-patch FC layer as the first block, with a patch size of $P = 4$. The input token size was fixed at $S_0 = (32/P)^2 = 64$. We trained the models on the CIFAR-10, CIFAR-100, and STL-10 datasets, along with data augmentations such as random cropping and random horizontal flipping. The input images were resized to a size of $32 \times 32 \times 3$. We employed Nesterov SGD with a mini-batch size 128 and a momentum of 0.9 for training, running for 200 epochs. The initial learning rate was set to 0.02, and we used cosine annealing for learning rate scheduling. To ensure robustness, we conducted three trials for each configuration and reported the mean and standard deviation of the results. Unless otherwise specified, these settings were used throughout the whole study on CIFAR-10, CIFAR-100, and STL-10.

(i) S-Mixer and RP S-Mixer We conducted training experiments on the S-Mixer architecture with eight blocks. In order to explore various cases of integer pairs (S, C) that approximately satisfy the equation

$$\Omega = \frac{CS^2 + SC^2}{2}. \tag{S.42}$$

The number of connections, denoted as Ω, was fixed at $\Omega = 2^{18}, 2^{21}$ and 2^{27}. For each value of Ω, the pairs (C, S) were chosen in a symmetric manner. It should be noted that if $(C, S) = (a, b)$ is a solution, then $(C, S) = (b, a)$ is also a solution. The selected pairs for each value of Ω are as follows:

- $\Omega = 2^{18}$: $(C, S) = (16, 173), (32, 113), (48, 83), (83, 48), (113, 32), (173, 16)$.
- $\Omega = 2^{21}$: $(C, S) = (16, 504), (32, 346), (64, 226), (226, 64), (346, 32), (504, 16)$.
- $\Omega = 2^{27}$: $(C, S) = (128, 1386), (256, 904), (904, 256), (1386, 128)$.

(ii) SW-MLP. For a fair comparison between the Mixers and SW-MLPs, we set the first layer of both models to the same per-patch FC structure. We trained SW-MLPs with eight blocks, where the hidden layers of these MLPs share a common $\Omega = 2^{18}$ connectivity pattern. Following the per-patch fully connected layer, the feature matrix is vectorized and processed as a standard MLP with masked sparse connectivity.

For each freezing rate $1 - p$, we determined the width m of the hidden units using the equation:

$$\Omega = m^2 p, \quad m = \sqrt{\frac{\Omega}{p}}. \tag{S.43}$$

We set $1 - p = 0.1, 0.3, 0.5, 0.7, 0.9$, which correspond to $m = 540, 612, 724, 935, 1619$, respectively.

C.5. Figure 6

We set the number of blocks L and the other hyper-parameters as in Table S.2 .

dataset	L	Ω	γ	optimizer	init. lr	mini-batch	epoch	hard aug.
CIFAR10	12	2^{21}	None	AdamW	0.02	1024	2000	✓
CIFAR100	12	2^{21}	None	SGD	0.1	128	600	
STL10	12	2^{18}	None	SGD	0.1	128	2000	
ImageNet-1k	8	290217984	4	AdamW	0.001	4096	300	✓

Table S.2 . Hyper-parameters of models for normal/RP Mixers in Figure 6.

CIFAR-10, 100, STL-10. We utilized Tesla V100 GPUs and approximately 200 GPU hours for our experiments. We used a single GPU per each run. We set $\Omega = 2^{18}$ and used the same pairs (S, C) satisfying equation S.42 as Sec. C.4.

We set the initial learning rate to 0.1. On CIFAR-10, we trained models 200 epochs, 600-epochs on CIFAR-100, and we did five trials with random seeds. We trained models 2000-epochs on STL-10 with three random seeds.

ImageNet-1k. We utilized Tesla V100 GPUs and approximately 4000 GPU hours for our experiments. for training MLP-Mixer and RP MLP-Mixer on ImageNet-1k; we used a GPU cluster of 32 nodes of 4 GPUs per node for each run.

We set the expansion factor $\gamma = 4$ for both token-mixing MLP and channel-mixing MLP. We set $\Omega = 290217984 = (768^2 \cdot 196 + 768 \cdot 196^2)\gamma/2$ on a baseline $P = 16, (S, C) = (196, 768)$. We sweep $P = 7, 8, 14, 16, 28, 32$ and set $C = 3P^2$ and set S so that it approximately satisfies the equation

$$\Omega = \frac{\gamma(CS^2 + SC^2)}{2}. \tag{S.44}$$

For each setting, we did three trials with random seeds.

Training on the ImageNet-1k is based on the timm library (Wightman, 2019). We used AdamW with an initial learning rate of 10^{-3} and 300 epochs. We set the mini-batch size to 4096 and used data-parallel training with a batch size of 32 in each GPU. We use the warm-up of with the warm-up learning rate 10^{-6} and the warm-up epoch 5. We used the cosine annealing of the learning rate with a minimum learning rate 10^{-5}. We used the weight-decay 0.05. We applied the random erasing in images with a ratio of 0.25. We also applied the random auto-augmentation with a policy rand-m9-mstd0.5-inc1. We used the mix-up with $\alpha = 0.8$ and the cut-mix with $\alpha = 1.0$ by switching them in probability 0.5. We used the label smoothing with $\varepsilon = 0.1$.

C.6. Figure 7

For our experiments in Figure 7, we utilized Tesla V100 GPUs, with approximately 70 GPU hours utilized. We trained both S-Mixer and RP S-Mixer models on CIFAR-10, CIFAR-100, and STL-10 datasets. We considered different numbers of blocks, specifically $L = 4, 8, 12, 16, 20$. The values of S and C were fixed at 128. Each configuration was evaluated using three trials with different random seeds.

D. Supplementary Experiments

D.1. Trainability of highly sparse weights

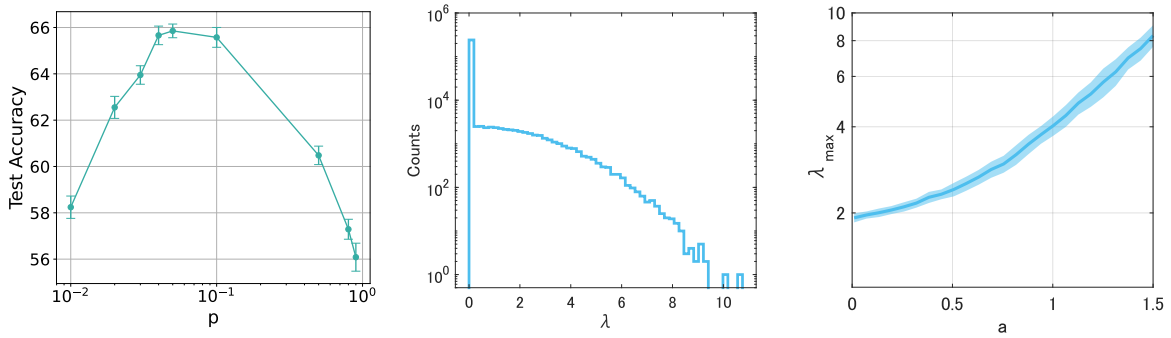

Figure S.1 . On the trainability of SW-MLP. (Left) Trainability decreases as the sparsity $1 - p$ becomes too high. We set γ according to (Golubeva et al., 2021) under fixed $\Omega = 2^{16}$. We performed five trials for each p with random seeds. (Center) The singular value distribution of the sparse weight at random initialization. We set $a = 1.5$, $\Omega = 10^3$ and performed 50 trials. (Right) The largest eigenvalue monotonically increases as the sparsity increases.

Golubeva et al. (2021) found that as the sparsity (width) increased to some extent, the generalization performance improved. They also reported that if the sparsity became too high, the generalization performance slightly decreased. They discussed that this decrease was caused by the deterioration of trainability, that is, it became difficult for the gradient descent to decrease the loss function. In fact, we confirmed their decrease of the performance in SW-MLP as is shown in Figure S.1 (left). In contrast, we hardly observed such a decrease in performance for the Mixers. This seems rational because we can take an arbitrary small sparsity $1 - p$ for the SW-MLP while it is lower-bounded for the Mixers as is described in Section 4.

As a side note, we give here quantitative insight into the trainability from the perspective of the singular value of the weight matrix. Some previous work reported that the large singular values of weight matrices at random initialization cause the deterioration of trainability in deep neural networks (Bjorck et al., 2018). Following this line of study, let us consider a random weight of SW-MLP. Set the width by $m = \Omega^{(1+a)/2}$ and the sparsity by $p = 1/\Omega^a$ with a constant $a > 0$ and take a large Ω limit. We use this scaling because our interest is in the case where the expected number of weights, i.e., $m^2 p = \Omega$, is independent of the scaling of p. We generate $Z = M \odot W$ where $W_{ij} \sim \mathcal{N}(0, 1)$ and M is a static mask matrix whose entries are given by the Bernoulli distribution with probability p. The singular value of the weight matrix Z is equivalent to the square root of the eigenvalue of $Q = ZZ^\top$. Because we are uninterested in a trivial scale factor, we scaled the matrix Q as Q/c where c denotes the average over the diagonal entries of Q. This makes the trace of Q, that is, the summation (or average) of eigenvalues, a constant independent of a. We computed the eigenvalues of Q and obtained the singular values of Z denoted by λ.